Algorithm Construction and Analysis

Complete Study Guide for Examination Topics 2025/26
(Minimal Level Questions)

Gemini Notebook

September 6, 2026

Contents

Chapter 1

Advanced Data Structures

1.1 Question 1: Trie (Prefix Tree) - Concept, Properties, Operations, Applications, and Examples

1.1.1 Concept

A Trie, also known as a prefix tree, is an ordered tree-like data structure used to store a dynamic set of keys, typically strings. Unlike a standard binary search tree, nodes in a Trie do not store the key itself; instead, their position in the tree defines the associated key. The root node represents an empty string, and every edge descending from a node is labeled with a character from the alphabet. The path from the root to any given node u defines a prefix of one or more strings stored in the Trie.

1.1.2 Properties

- **Prefix Sharing**: All descendants of a node share a common prefix represented by the path from the root to that node.
- **Height Bound**: The maximum height of the Trie is equal to the length of the longest string stored in it (L).
- **Alphabet-Bounded Degree**: Each node has at most Σ children, where Σ is the size of the alphabet.
- **Key Termination**: Since a prefix of a word can also be a valid word, nodes contain a boolean flag (`is_end_of_word`) to indicate whether the path from the root to that node represents a complete stored word.

1.1.3 Basic Operations

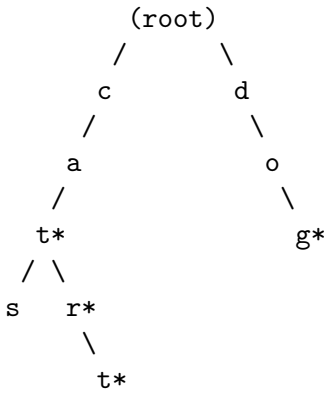
1. **Insertion**: To insert a string, start from the root and traverse down the tree following characters of the string. If a child node for a character does not exist, create a new node. Mark the final node's `is_end_of_word` as true.
2. **Search**: Traverse from the root following characters of the target string. If at any step the required child is missing, the string is not in the Trie. If traversal succeeds, return true if the final node has `is_end_of_word` set to true.
3. **Prefix Search (StartsWith)**: Similar to Search, but returns true if the traversal completes, regardless of the `is_end_of_word` flag of the final node.

1.1.4 Applications

- **Autocomplete / Predictive Text**: Efficiently retrieving all words with a given prefix.
- **Spell Checkers**: Rapidly verifying if a word exists in a dictionary.
- **IP Routing (Longest Prefix Match)**: Finding the longest matching routing prefix.
- **T9 Text Input**: Translating keypress sequences to candidate words.

1.1.5 Concrete Example

Suppose we insert the following strings into an initially empty Trie: `"cat"`, `"car"`, `"cart"`, `"dog"`.



In this diagram, nodes with an asterisk (*) represent valid words (i.e., `is_end_of_word = true`). Note how `"cat"`, `"car"`, and `"cart"` share the prefix `"ca"`.

1.2 Question 2: Trie (Prefix Tree) - C++ Implementation, Complexity, and Comparison with Balanced BSTs and Hash Tables

1.2.1 C++ Pseudocode / Implementation

Below is a standard C++ implementation of a Trie node and class utilizing an array for fixed-size alphabets (lowercase English letters, $\Sigma = 26$).

```cpp
#include <iostream>
#include <string>

struct TrieNode {
    TrieNode* children[26];
    bool is_end_of_word;

    TrieNode() {
        is_end_of_word = false;
        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};

class Trie {
private:
    TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }

    void insert(const std::string& word) {
        TrieNode* curr = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (curr->children[idx] == nullptr) {
                curr->children[idx] = new TrieNode();
            }
            curr = curr->children[idx];
        }
        curr->is_end_of_word = true;
    }

    bool search(const std::string& word) {
        TrieNode* curr = root;
        for (char ch : word) {
            int idx = ch - 'a';
            if (curr->children[idx] == nullptr) {
                return false;
            }
            curr = curr->children[idx];
        }
        return curr->is_end_of_word;
    }

    bool startsWith(const std::string& prefix) {
```

```
        TrieNode* curr = root;
        for (char ch : prefix) {
            int idx = ch - 'a';
            if (curr->children[idx] == nullptr) {
                return false;
            }
            curr = curr->children[idx];
        }
        return true;
    }
};
```

1.2.2 Complexity Analysis

- **Time Complexity**:
  - `insert`: $O(L)$, where L is the length of the string being inserted. We perform exactly L character transitions.
  - `search`: $O(L)$, as we perform at most L character transitions.
  - `startsWith`: $O(L)$, requiring at most L transitions.

  The time complexity depends strictly on the key length L and is completely independent of the number of elements N stored in the Trie.
- **Space Complexity**: In the worst case, the space complexity is $O(M \cdot \Sigma)$, where M is the sum of the lengths of all strings inserted (total states) and Σ is the alphabet size (26 in this case). While Trie shares prefixes, the memory overhead of null pointers per node can be significant. This can be optimized using a hash map or dynamic vector for node children instead of fixed arrays.

1.2.3 Comparison Table

Below is a comparison of Tries, Balanced Binary Search Trees (e.g., AVL tree, Red-Black tree as in `std::set`), and Hash Tables (e.g., `std::unordered_set`) for storing N strings of average length L.

Metric	Trie	Balanced BST	Hash Table
Search Time (Worst Case)	$O(L)$	$O(L \log N)$	$O(L \cdot N)$ (due to co
Search Time (Average Case)	$O(L)$	$O(L \log N)$	$O(L)$
Insertion Time	$O(L)$	$O(L \log N)$	$O(L)$ (average)
Prefix Search Support	$O(L)$ (optimal)	$O(L \log N)$ (using bounds)	Not supported nati
Space Overhead	$O(M \cdot \Sigma)$	$O(N \cdot L)$	$O(N \cdot L)$
Key Ordering	Ordered (Lexicographical)	Ordered	Unordered

Table 1.1: Algorithmic comparison of string storage structures

1.3 Question 3: Disjoint Sets (Union-Find) - Efficient Structure, Simple Structure Comparison, Implementation, and Examples

1.3.1 Concept and Operations

The Disjoint-Set (Union-Find) data structure maintains a collection of disjoint dynamic sets $\mathcal{S} = \{S_1, S_2, \ldots, S_k\}$. Each set is identified by a representative element, which is typically some member of the set. The two fundamental operations are:

1. `find(x)`: Returns the representative of the unique set containing element x.
2. `union(x, y)`: Merges the set containing x and the set containing y into a single set.

1.3.2 Naive vs. Efficient Structure

- **Simple/Naive Approach**: Represent sets using a flat array `group_id[i]` storing the partition ID of each element.
  - `find(x)`: Simply return `group_id[x]` in $O(1)$ time.
  - `union(x, y)`: Iterate through the entire array and change all occurrences of `group_id[x]` to `group_id[y]`. This takes $O(N)$ time.
- **Efficient Approach (Forest of Trees)**: Represent sets as trees, where each node points to its parent. The root of the tree is the representative. Two crucial optimizations are applied:
  1. **Union by Rank / Size**: Always attach the root of the smaller tree to the root of the larger tree. This prevents the tree from degenerating into a linked list, keeping the height bounded by $O(\log N)$.
  2. **Path Compression**: During `find(x)`, make all nodes on the path from x to the root point directly to the root. This flattens the tree dynamically.

  Combining both optimizations reduces the amortized time complexity per operation to $O(\alpha(N))$, where α is the extremely slowly growing inverse Ackermann function (practically constant, $\alpha(N) \leq 4$ for all realistic inputs).

1.3.3 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <numeric>

class UnionFind {
private:
    std::vector<int> parent;
    std::vector<int> rank;

public:
    UnionFind(int n) {
        parent.resize(n);
        iota(parent.begin(), parent.end(), 0); // parent[i] = i
        rank.assign(n, 0);                     // rank[i] = 0
    }

    int find(int x) {
```

```
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // Path compression
        }
        return parent[x];
    }

    bool unite(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX == rootY) return false;

        // Union by rank
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
        return true;
    }
};
```

1.3.4 Trace Example

Let we have 5 elements $\{0, 1, 2, 3, 4\}$ initially in their own sets:

1. Initial: Parent array: `[0, 1, 2, 3, 4]`, Rank array: `[0, 0, 0, 0, 0]`.
2. `unite(0, 1)`: Roots: 0 and 1. Rank(0) = Rank(1) = 0. We make 1 point to 0. Rank(0) becomes 1. Parent: `[0, 0, 2, 3, 4]`.
3. `unite(2, 3)`: Roots: 2 and 3. Rank(2) = Rank(3) = 0. We make 3 point to 2. Rank(2) becomes 1. Parent: `[0, 0, 2, 2, 4]`.
4. `unite(1, 3)`: Find(1) returns 0. Find(3) returns 2. Rank(0) = 1, Rank(2) = 1. We make 2 point to 0. Rank(0) becomes 2. Parent: `[0, 0, 0, 2, 4]`.
5. `find(3)`: Since parent[3] = 2, we recurse to parent[2] = 0. On return, path compression updates parent[3] to 0 directly. Parent array becomes: `[0, 0, 0, 0, 4]`.

Chapter 2

Static and Dynamic Range Queries

2.1 Question 4: Static Range Queries - Concept and Fundamental Techniques

2.1.1 Concept

A range query problem asks us to compute some associative operation (e.g., Sum, Min, Max, GCD) over a contiguous subarray $A[L..R]$ of an array A of size N. In a **static** setting, the elements of the array A never change after initialization. This allows us to perform substantial preprocessing to answer queries in $O(1)$ or $O(\log N)$ time.

2.1.2 Technique 1: Prefix Sum Array (for Range Sum Queries)

For range sum queries, the optimal technique is the Prefix Sum Array. We construct an auxiliary array P of size $N + 1$ where:

$$P[i] = \sum_{j=0}^{i-1} A[j], \quad P[0] = 0$$

Any range sum query $\sum_{j=L}^{R} A[j]$ can be answered in $O(1)$ time as:

$$\text{Query}(L, R) = P[R + 1] - P[L]$$

- **Preprocessing Time**: $O(N)$
- **Query Time**: $O(1)$
- **Space Complexity**: $O(N)$

2.1.3 Technique 2: Sparse Table (for Range Minimum Queries - RMQ)

For static range queries where the operator is idempotent (i.e., $x \circ x = x$ such as Min, Max, GCD), we use a Sparse Table. We define a 2D table M of size $N \times \lfloor \log_2 N \rfloor$. The entry $M[i][j]$ stores the result of the query in the range starting at index i of length 2^j (i.e., range $[i, i+2^j-1]$).

- **Recurrence Relation**:

$$M[i][0] = A[i]$$
$$M[i][j] = \min(M[i][j-1], M[i + 2^{j-1}][j-1])$$

- **Query Answering**: For any arbitrary range $[L, R]$, let $k = \lfloor \log_2(R - L + 1) \rfloor$. Since the operation is idempotent, we can cover the range with two overlapping power-of-two intervals: $[L, L + 2^k - 1]$ and $[R - 2^k + 1, R]$.

$$\text{Query}(L, R) = \min(M[L][k], M[R - 2^k + 1][k])$$

- **Preprocessing Time**: $O(N \log N)$
- **Query Time**: $O(1)$
- **Space Complexity**: $O(N \log N)$

2.1.4 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <cmath>
#include <algorithm>

class SparseTable {
private:
    std::vector<std::vector<int>> st;
    std::vector<int> lg;

public:
    SparseTable(const std::vector<int>& A) {
        int n = A.size();
        int max_log = log2(n) + 1;
        st.assign(n, std::vector<int>(max_log));
        lg.assign(n + 1, 0);

        for (int i = 2; i <= n; i++) lg[i] = lg[i/2] + 1;

        for (int i = 0; i < n; i++) st[i][0] = A[i];

        for (int j = 1; j < max_log; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[i][j] = std::min(st[i][j-1], st[i + (1 << (j-1))][j
                    -1]);
            }
        }
    }

    int query(int L, int R) {
        int len = R - L + 1;
        int k = lg[len];
        return std::min(st[L][k], st[R - (1 << k) + 1][k]);
    }
};
```

2.2 Question 5: Segment Trees - Concept, Properties, and Construction (Bottom-Up vs. Top-Down)

2.2.1 Concept

A Segment Tree is a versatile, dynamic binary tree data structure used to answer range queries and perform element updates in logarithmic time. Each node in a Segment Tree represents a contiguous interval $[L, R]$ of the underlying array. The root node represents the entire array range $[0, N-1]$.

2.2.2 Properties

- **Structure**: If a node represents the interval $[L, R]$, and $L < R$, its left child represents $[L, M]$ and its right child represents $[M+1, R]$, where $M = \lfloor (L+R)/2 \rfloor$. If $L = R$, the node is a leaf representing a single array element $A[L]$.
- **Tree Size**: For an array of size N, the height of the segment tree is $O(\log N)$. The total number of nodes in a complete binary tree of this form is bounded by $2^{\lceil \log_2 N \rceil + 1} - 1 < 4N$.
- **Dynamic Operations**: Unlike a static Sparse Table, Segment Trees allow both point updates (changing one element) and range queries in $O(\log N)$ time.

2.2.3 Construction: Top-Down vs. Bottom-Up

There are two main strategies to build a Segment Tree from an array A:

1. **Top-Down (Recursive)**: We start at the root node and recursively split the current interval $[L, R]$ into left and right halves until we reach leaf nodes ($L = R$). After the children return their values, we merge them to compute the parent's value. Time Complexity: $O(N)$ because there are $O(N)$ nodes and we spend $O(1)$ time per node.
2. **Bottom-Up (Iterative)**: We store the segment tree in a flat array of size $2N$. The original elements are placed directly at indices N to $2N-1$ (leaves). Then, we populate parent nodes at indices $N-1$ down to 1 by combining their left child (`2*i`) and right child (`2*i+1`). Time Complexity: $O(N)$ with extremely low constant factors and no recursion overhead.

2.2.4 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <iostream>

// 1. Top-Down Representation (Array of size 4N)
class SegmentTreeTopDown {
private:
    int n;
    std::vector<int> tree;

    void build(const std::vector<int>& A, int node, int L, int R) {
        if (L == R) {
            tree[node] = A[L];
            return;
        }
        int M = L + (R - L) / 2;
        build(A, 2 * node, L, M);
        build(A, 2 * node + 1, M + 1, R);
```

```
        tree[node] = tree[2 * node] + tree[2 * node + 1]; // Merge step
    }

public:
    SegmentTreeTopDown(const std::vector<int>& A) {
        n = A.size();
        tree.assign(4 * n, 0);
        build(A, 1, 0, n - 1);
    }
};

// 2. Bottom-Up Representation (Array of size 2N)
class SegmentTreeBottomUp {
private:
    int n;
    std::vector<int> tree;

public:
    SegmentTreeBottomUp(const std::vector<int>& A) {
        n = A.size();
        tree.assign(2 * n, 0);
        // Step 1: Copy leaves
        for (int i = 0; i < n; i++) {
            tree[n + i] = A[i];
        }
        // Step 2: Build parents
        for (int i = n - 1; i > 0; i--) {
            tree[i] = tree[2 * i] + tree[2 * i + 1];
        }
    }
};
```

2.3 Question 6: Segment Trees - Range Sum Query (Bottom-Up vs. Top-Down) with Trace Examples

2.3.1 Top-Down Range Query

In the top-down recursive query, we start at the root node and compare the target query range $[qL, qR]$ with the node's interval $[L, R]$. There are three possible relations:

1. **No Overlap**: If $qL > R$ or $qR < L$, we return the identity element (0 for summation).
2. **Complete Overlap**: If $qL \leq L$ and $R \leq qR$, we return the precomputed value stored in the node.
3. **Partial Overlap**: If there is a partial intersection, we recurse into the left and right children and sum their results.

2.3.2 Bottom-Up Range Query

In the bottom-up iterative approach, we map the query boundaries L and R to their corresponding leaf indices: $l = L + N$ and $r = R + N$. We then traverse upwards while $l \leq r$:

- If l is a right child (`l & 1`), then the interval covered by its parent extends outside the left boundary. We include `tree[l]` individually and increment l to move inside.
- If r is a left child (`!(r & 1)`), then the interval of its parent extends outside the right boundary. We include `tree[r]` individually and decrement r to move inside.
- Finally, we move both pointers to their parents: $l = l/2$ and $r = r/2$.

2.3.3 C++ Pseudocode / Implementation

```cpp
// 1. Top-Down Query Method
int queryTopDown(int node, int L, int R, int qL, int qR, const std::
    vector<int>& tree) {
    if (qL <= L && R <= qR) return tree[node]; // Complete overlap
    if (qR < L || qL > R) return 0;            // No overlap
    int M = L + (R - L) / 2;
    return queryTopDown(2 * node, L, M, qL, qR, tree) +
           queryTopDown(2 * node + 1, M + 1, R, qL, qR, tree); //
               Partial
}

// 2. Bottom-Up Query Method
int queryBottomUp(int L, int R, int n, const std::vector<int>& tree) {
    int sum = 0;
    for (L += n, R += n; L <= R; L /= 2, R /= 2) {
        if (L & 1) sum += tree[L++];
        if (!(R & 1)) sum += tree[R--];
    }
    return sum;
}
```

2.3.4 Trace Example

Let array $A = [1, 3, 5, 7, 9, 11]$ ($N = 6$). The bottom-up tree size is 12: Leaves at indices 6..11: `tree[6..11] = [1, 3, 5, 7, 9, 11]`. Parents:

- `tree[5] = tree[10] + tree[11] = 9 + 11 = 20`
- `tree[4] = tree[8] + tree[9] = 5 + 7 = 12`
- `tree[3] = tree[6] + tree[7] = 1 + 3 = 4`
- `tree[2] = tree[4] + tree[5] = 12 + 20 = 32`
- `tree[1] = tree[2] + tree[3] = 32 + 4 = 36`

Let's query sum of range $[1, 4]$ (representing values $[3, 5, 7, 9]$, sum = 24).

- Initialize: $L = 1 + 6 = 7$, $R = 4 + 6 = 10$.
- Step 1: $L = 7, R = 10$. $L \leq R$.
  - $L\&1 = 7\&1 = 1 \implies$ sum += tree[7] (3). L becomes 8.
  - $!(R\&1) = !(10\&1) = 1 \implies$ sum += tree[10] (9). R becomes 9.
  - Parents: $L = 8/2 = 4$, $R = 9/2 = 4$.
- Step 2: $L = 4, R = 4$. $L \leq R$.
  - L is even (not right child).
  - R is even (not left child) $\implies$ sum += tree[4] (12). R becomes 3.
  - Parents: $L = 2, R = 1$.
- Loop ends as $L > R$. Total sum $= 3 + 9 + 12 = 24$.

2.4 Question 7: Segment Trees - Point Update (Bottom-Up vs. Top-Down) with Trace Examples

2.4.1 Top-Down Point Update

In the top-down point update, we recursively traverse the segment tree, targeting the path leading to the index idx we want to update.

- If the current range $[L, R]$ does not contain idx, we immediately return.
- If we reach a leaf node ($L = R = idx$), we update its value to val.
- On backtracking, each parent node recomputes its sum based on its newly updated children.
- Time Complexity: $O(\log N)$ as we visit exactly one node per tree level.

2.4.2 Bottom-Up Point Update

In the bottom-up approach, we update the leaf node directly at index $idx + N$. We then use a simple loop to walk directly up to the root, updating each parent node as the sum of its two children.

- Set `tree[idx + N] = val`.
- Set $i = idx + N$. While $i > 1$, set $i = i/2$ and `tree[i] = tree[2*i] + tree[2*i + 1]`.
- Time Complexity: $O(\log N)$ with no recursion overhead.

2.4.3 C++ Pseudocode / Implementation

```cpp
// 1. Top-Down Point Update Method
void updateTopDown(int node, int L, int R, int idx, int val, std::
    vector<int>& tree) {
    if (L == R) {
        tree[node] = val;
        return;
    }
    int M = L + (R - L) / 2;
    if (idx <= M) {
        updateTopDown(2 * node, L, M, idx, val, tree);
    } else {
        updateTopDown(2 * node + 1, M + 1, R, idx, val, tree);
    }
    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

// 2. Bottom-Up Point Update Method
void updateBottomUp(int idx, int val, int n, std::vector<int>& tree) {
    int i = idx + n;
    tree[i] = val;
    for (i /= 2; i > 0; i /= 2) {
        tree[i] = tree[2 * i] + tree[2 * i + 1];
    }
}
```

2.4.4 Trace Example

Let's update element at index 2 (currently 5) to 6 in our bottom-up segment tree of $N = 6$.

- Tree leaves indices 6..11: `[1, 3, 5, 7, 9, 11]`. Node 2 is stored at tree index $2+6 = 8$.
- Step 1: Update tree leaf `tree[8] = 6`.
- Step 2: $i = 8/2 = 4$.

$$\texttt{tree[4]} = \texttt{tree[8]} + \texttt{tree[9]} = 6 + 7 = 13 \quad \text{(was 12)}$$

- Step 3: $i = 4/2 = 2$.

$$\texttt{tree[2]} = \texttt{tree[4]} + \texttt{tree[5]} = 13 + 20 = 33 \quad \text{(was 32)}$$

- Step 4: $i = 2/2 = 1$.

$$\texttt{tree[1]} = \texttt{tree[2]} + \texttt{tree[3]} = 33 + 4 = 37 \quad \text{(was 36)}$$

- Update complete in exactly 3 iterations.

Chapter 3

Graph Representations and Fundamental Traversals

3.1 Question 8: Graphs - Basic Concepts, Representation of Unweighted and Weighted Graphs, and Complexity Analysis

3.1.1 Basic Concepts

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges $E \subseteq V \times V$. Graphs can be:

- **Directed**: Edges have a direction (ordered pairs).
- **Undirected**: Edges are unordered pairs.
- **Weighted**: Each edge has an associated numerical value (weight).
- **Unweighted**: All edges are considered to have uniform cost.

3.1.2 Graph Representations

1. **Adjacency Matrix**: A 2D array `adj[V][V]` where `adj[u][v]` is 1 (or edge weight for weighted graphs) if there is an edge from u to v, and 0 (or ∞) otherwise.
2. **Adjacency List**: An array of size V, where each element is a list of neighbors of vertex u. For weighted graphs, the list stores pairs of `(neighbor, weight)`.
3. **Edge List**: A simple collection of all edges in the graph, usually represented as a vector of structures or tuples `(u, v)` or `(u, v, weight)`.

3.1.3 Complexity Analysis

Let V be the number of vertices and E the number of edges.

3.1.4 C++ Code for Representation

```cpp
#include <vector>

// 1. Weighted Adjacency Matrix
using AdjMatrix = std::vector<std::vector<int>>;

```

Operation / Property	**Adjacency Matrix**	**Adjacency List**	**Edge List**
Space Complexity	$O(V^2)$	$O(V+E)$	$O(E)$
Add Edge	$O(1)$	$O(1)$	$O(1)$
Remove Edge	$O(1)$	$O(\deg(u))$	$O(E)$
Check Edge (u, v)	$O(1)$	$O(\deg(u))$	$O(E)$
Find Neighbors of u	$O(V)$	$O(\deg(u))$	$O(E)$

Table 3.1: Complexity comparison of graph representations

```
// 2. Weighted Adjacency List
struct Edge {
    int to;
    int weight;
};
using AdjList = std::vector<std::vector<Edge>>;

// 3. Edge List
struct CompleteEdge {
    int from;
    int to;
    int weight;
};
using EdgeList = std::vector<CompleteEdge>;
```

3.2 Question 9: Graphs - Depth-First Search (DFS), Entry/Exit Processing, Correctness, and Connected Components

3.2.1 DFS Concept

Depth-First Search (DFS) is a fundamental traversal algorithm that explores a graph by going as deep as possible along each branch before backtracking. It uses a recursive stack to keep track of the path.

3.2.2 Entry and Exit Processing (DFS Timers)

We can track the discovery and finishing state of each vertex by maintaining a logical timer.

- `tin[u]` (Discovery time): The time at which vertex u is first colored gray (entered).
- `tout[u]` (Finishing time): The time at which all neighbors of u have been fully explored, and u is colored black (exited).

These timestamps are extremely useful for edge classification, topological sorting, and finding strongly connected components.

3.2.3 C++ Implementation

```cpp
#include <vector>
#include <iostream>

class DFSGraph {
private:
    int V;
    std::vector<std::vector<int>> adj;
    std::vector<int> visited; // 0: unvisited, 1: visiting (gray), 2: fully visited (black)
    std::vector<int> tin, tout;
    int timer;

    void dfs_visit(int u) {
        visited[u] = 1;
        tin[u] = ++timer;

        // Entry processing can happen here

        for (int v : adj[u]) {
            if (visited[v] == 0) {
                dfs_visit(v);
            }
        }

        visited[u] = 2;
        tout[u] = ++timer;
        // Exit processing can happen here
    }

public:
    DFSGraph(int vertices) : V(vertices), adj(vertices), visited(vertices, 0), tin(vertices), tout(vertices), timer(0) {}

    void addEdge(int u, int v) {
```

```
        adj[u].push_back(v);
    }

    void runDFS() {
        for (int i = 0; i < V; i++) {
            if (visited[i] == 0) {
                dfs_visit(i);
            }
        }
    }
};
```

3.2.4 Correctness of DFS

The correctness of DFS in visiting all reachable nodes is proven by the **White Path Theorem**: In a DFS forest of a graph G, vertex v is a descendant of vertex u if and only if at time `tin[u]` there is a path from u to v consisting entirely of unvisited (white) vertices.

3.2.5 Finding Connected Components (Undirected Graphs)

To find all connected components, we maintain a count and start a DFS traversal for each unvisited node. All nodes visited during a single DFS call belong to the same connected component.

```
int findConnectedComponents(int V, const std::vector<std::vector<int>>&
    adj, std::vector<int>& comp_id) {
    std::vector<bool> visited(V, false);
    comp_id.assign(V, -1);
    int count = 0;

    auto dfs = [&](auto& self, int u, int id) -> void {
        visited[u] = true;
        comp_id[u] = id;
        for (int v : adj[u]) {
            if (!visited[v]) {
                self(self, v, id);
            }
        }
    };

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(dfs, i, count);
            count++;
        }
    }
    return count; // Number of connected components
}
```

3.2.6 Complexity

- **Time Complexity**: $O(V + E)$ since we visit every vertex once and check every edge exactly once (or twice in undirected graphs).
- **Space Complexity**: $O(V)$ auxiliary space for the visited array and recursion stack.

3.3 Question 10: Graphs - DFS Tree, DFS Numbering, and Edge Classification of Undirected Graphs

3.3.1 DFS Tree and Forest

When running DFS on a graph, the paths we follow without backtracking form a spanning forest called the **DFS Spanning Forest**. If the graph is connected, it forms a single **DFS Spanning Tree**.

3.3.2 DFS Numbering

DFS numbering assigns discovery/finishing integers to vertices (`tin` and `tout`). This numbers represent structural parent-child relationships: u is an ancestor of v if and only if:

$$\texttt{tin[u]} < \texttt{tin[v]} < \texttt{tout[v]} < \texttt{tout[u]}$$

This is the parenthesization structure of tree intervals.

3.3.3 Edge Classification in Undirected Graphs

When we traverse an undirected graph, every edge can be classified into exactly one of two types based on the DFS traversal:

1. **Tree Edges**: Edges belonging to the DFS tree. An edge (u, v) is a tree edge if v was discovered (white) from u.
2. **Back Edges**: Edges (u, v) that are not tree edges, connecting a node u to its ancestor v in the DFS tree.

Note: There are **no forward or cross edges** in undirected graphs. An undirected edge can always be traversed from either direction; hence, any non-tree edge will be categorized as a back edge.

3.3.4 Trace Example

Let us take an undirected cycle of 4 vertices: $0 - 1 - 2 - 3 - 0$. Running DFS starting from 0:

1. Start at 0: `tin[0] = 1`. Neighbors: 1, 3. Visited is `[1, 0, 0, 0]`.
2. Move to 1: Edge $0 \to 1$ is a **Tree Edge**. `tin[1] = 2`. Neighbors: 0 (visited), 2. Visited: `[1, 1, 0, 0]`.
3. Move to 2: Edge $1 \to 2$ is a **Tree Edge**. `tin[2] = 3`. Neighbors: 1 (visited), 3. Visited: `[1, 1, 1, 0]`.
4. Move to 3: Edge $2 \to 3$ is a **Tree Edge**. `tin[3] = 4`. Neighbors: 2 (visited), 0 (visited). Visited: `[1, 1, 1, 1]`.
5. At 3, examine neighbor 0. Since 0 is visited and is currently on the active stack (`visited[0] = 1`), edge $3 \to 0$ is classified as a **Back Edge**.
6. Backtrack:
   - Exit 3: `tout[3] = 5`.
   - Exit 2: `tout[2] = 6`.
   - Exit 1: `tout[1] = 7`.
   - Exit 0: `tout[0] = 8`.

3.4 Question 11: Graphs - DFS Classification/Characterization of Directed Graphs and Cycle Detection

3.4.1 Edge Classification in Directed Graphs

In a directed graph, DFS splits edges into four distinct classes:

1. **Tree Edges**: Directed edges (u, v) where v was first discovered via edge (u, v).
2. **Back Edges**: Edges (u, v) where v is an ancestor of u in the DFS tree. This includes self-loops.
3. **Forward Edges**: Non-tree edges (u, v) where v is a descendant of u in the DFS tree.
4. **Cross Edges**: All other edges. These connect vertices in different subtrees of the DFS forest or different branches of the same tree.

3.4.2 Mathematical Characterization using DFS Timers

We can classify any directed edge (u, v) immediately using the discovery time `tin` and finishing time `tout`:

- **Tree / Forward Edge**: `tin[u]` $<$ `tin[v]` $<$ `tout[v]` $<$ `tout[u]`. (To distinguish Tree from Forward, check if u is the direct parent of v in the tree).
- **Back Edge**: `tin[v]` $\leq$ `tin[u]` $<$ `tout[u]` $\leq$ `tout[v]`.
- **Cross Edge**: `tin[v]` $<$ `tout[v]` $<$ `tin[u]` $<$ `tout[u]`.

3.4.3 Cycle Detection in Directed Graphs

A directed graph contains a cycle if and only if a DFS traversal encounters at least one **Back Edge**. We implement this by maintaining three states for each node during DFS:

- `State 0 (White)`: Unvisited.
- `State 1 (Gray)`: Currently visiting (active on recursion stack).
- `State 2 (Black)`: Fully processed.

If we see an edge to a Gray node, we have detected a Back Edge, confirming the existence of a cycle.

3.4.4 C++ Code for Cycle Detection

```cpp
#include <vector>

bool dfs_has_cycle(int u, const std::vector<std::vector<int>>& adj, std
    ::vector<int>& state) {
    state[u] = 1; // Mark gray (visiting)
    for (int v : adj[u]) {
        if (state[v] == 1) {
            return true; // Encountered a gray node => Back Edge found!
        }
        if (state[v] == 0) {
            if (dfs_has_cycle(v, adj, state)) return true;
        }
    }
```

```
    state[u] = 2; // Mark black (fully visited)
    return false;
}

bool containsCycle(int V, const std::vector<std::vector<int>>& adj) {
    std::vector<int> state(V, 0);
    for (int i = 0; i < V; i++) {
        if (state[i] == 0) {
            if (dfs_has_cycle(i, adj, state)) return true;
        }
    }
    return false;
}
```

3.5 Question 12: Graphs - Breadth-First Search (BFS), BFS Tree, Numbering, and Complexity

3.5.1 BFS Concept

Breadth-First Search (BFS) is a level-by-level traversal algorithm. Starting from a source node s, it explores all direct neighbors of s first, then all neighbors of neighbors, and so on. It uses a FIFO (First-In, First-Out) Queue to maintain the traversal order.

3.5.2 BFS Spanning Tree and BFS Numbering

- **BFS Tree**: The tree formed by edges that lead to the first discovery of each vertex. It represents shortest paths in terms of the number of edges from the source vertex in unweighted graphs.
- **BFS Numbering**: The sequential discovery order of vertices, or their minimal distance (layer number) from the start vertex.

3.5.3 C++ Implementation

```cpp
#include <vector>
#include <queue>
#include <iostream>

void runBFS(int source, const std::vector<std::vector<int>>& adj, std::vector<int>& dist, std::vector<int>& parent) {
    int V = adj.size();
    dist.assign(V, -1);
    parent.assign(V, -1);

    std::queue<int> q;
    dist[source] = 0;
    q.push(source);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] == -1) { // Node is unvisited
                dist[v] = dist[u] + 1;
                parent[v] = u; // Tree edge (u, v)
                q.push(v);
            }
        }
    }
}
```

3.5.4 Complexity Analysis

- **Time Complexity**: $O(V + E)$ since each node is queued at most once and each edge is processed once (or twice in undirected graphs).
- **Space Complexity**: $O(V)$ for the queue and helper arrays.

3.5.5 Trace Example

Let's run BFS starting at vertex 0 on an undirected cycle of 4 vertices: $0 - 1 - 2 - 3 - 0$.

1. Start: `dist = [0, -1, -1, -1]`, `parent = [-1, -1, -1, -1]`, Queue: `[0]`.
2. Dequeue 0. Neighbors: 1, 3. Both unvisited.
   - Update 1: `dist[1] = 1`, `parent[1] = 0`, Queue: `[1]`.
   - Update 3: `dist[3] = 1`, `parent[3] = 0`, Queue: `[1, 3]`.
3. Dequeue 1. Neighbors: 0 (visited), 2 (unvisited).
   - Update 2: `dist[2] = 2`, `parent[2] = 1`, Queue: `[3, 2]`.
4. Dequeue 3. Neighbors: 0 (visited), 2 (visited). No updates. Queue: `[2]`.
5. Dequeue 2. Neighbors: 1 (visited), 3 (visited). No updates. Queue empty.
6. Final distances: `[0, 1, 2, 1]`, Parent tree: `parent = [-1, 0, 1, 0]`.

Chapter 4

Advanced Graph Algorithms

4.1 Question 13: Graphs - Topological Sorting: Kahn's and DFS-Based Algorithms

4.1.1 Topological Sorting Concept

A Topological Sort of a Directed Acyclic Graph (DAG) is a linear ordering of its vertices such that for every directed edge $u \to v$, vertex u comes before v in the ordering. If the graph contains any cycle, a topological ordering is mathematically impossible.

4.1.2 Consecutive Algorithms

As requested, we present both fundamental topological sorting algorithms consecutively.

Algorithm 1: Kahn's Algorithm (In-Degree Queue-Based)

Kahn's algorithm is based on tracking the in-degree (number of incoming edges) of each node:

1. Compute the in-degree of all vertices.
2. Push all vertices with an in-degree of 0 into a FIFO queue.
3. While the queue is not empty, dequeue u, add it to the topological order, and decrement the in-degree of all neighbors v. If any neighbor's in-degree drops to 0, push it to the queue.
4. If the final order contains fewer than V vertices, the graph must contain a cycle.

Algorithm 2: DFS-Based Topological Sort

The DFS-based approach is based on discovery and finishing times:

1. Run DFS on the graph.
2. When a vertex u has finished exploring all of its outgoing branches (i.e., we are about to exit `dfs_visit(u)`), push u to a stack or prepend it to our list.
3. If we detect any Back Edge during DFS, the graph has a cycle.
4. The final reversed finishing times give the topological order.

4.1.3 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <queue>
#include <algorithm>

// Kahn's Algorithm
bool kahnsTopologicalSort(int V, const std::vector<std::vector<int>>& adj, std::vector<int>& order) {
    std::vector<int> in_degree(V, 0);
    for (int u = 0; u < V; u++) {
        for (int v : adj[u]) in_degree[v]++;
    }

    std::queue<int> q;
    for (int i = 0; i < V; i++) {
        if (in_degree[i] == 0) q.push(i);
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);

        for (int v : adj[u]) {
            in_degree[v]--;
            if (in_degree[v] == 0) q.push(v);
        }
    }
    return order.size() == V; // Returns true if DAG, false if cycle exists
}

// DFS-Based Algorithm
bool dfs_top_sort(int u, const std::vector<std::vector<int>>& adj, std::vector<int>& state, std::vector<int>& order) {
    state[u] = 1; // visiting
    for (int v : adj[u]) {
        if (state[v] == 1) return false; // Cycle detected
        if (state[v] == 0) {
            if (!dfs_top_sort(v, adj, state, order)) return false;
        }
    }
    state[u] = 2; // fully visited
    order.push_back(u);
    return true;
}

bool dfsTopologicalSort(int V, const std::vector<std::vector<int>>& adj, std::vector<int>& order) {
    std::vector<int> state(V, 0);
    order.clear();
    for (int i = 0; i < V; i++) {
        if (state[i] == 0) {
            if (!dfs_top_sort(i, adj, state, order)) return false;
        }
    }
    std::reverse(order.begin(), order.end());
```

```
53      return true;
54  }
```

4.2 Question 14: Graphs - Strongly Connected Components (SCC): Naive, Kosaraju, and Tarjan Algorithms

4.2.1 Strongly Connected Components (SCC) and Condensation

A Strongly Connected Component (SCC) of a directed graph G is a maximal subgraph $C \subseteq G$ such that for every pair of vertices $u, v \in C$, there is a path from u to v and a path from v to u. The **Condensation Graph** G^{SCC} is a directed acyclic graph formed by collapsing each individual SCC into a single super-node.

4.2.2 Consecutive Algorithms

We present all three alternative approaches for SCC consecutively, from naive to optimal.

Algorithm 1: Naive Reachability Intersection ($O(V \cdot (V + E))$)

We run a search (DFS/BFS) from every node u to find all nodes reachable from u. We then check which of those nodes can also reach u.

1. For each vertex u, find its reachable set $R(u)$ using DFS.
2. Vertices u and v are in the same SCC if and only if $v \in R(u)$ and $u \in R(v)$.
3. This is highly inefficient but conceptually simple.

Algorithm 2: Kosaraju's Double-DFS Algorithm ($O(V + E)$)

Kosaraju's algorithm uses the property that the condensation graph is a DAG, and the transpose graph G^T (graph with all edges reversed) shares the exact same SCCs as G:

1. Run DFS on G and push nodes to a stack based on their finishing times.
2. Compute the transpose graph G^T.
3. While the stack is not empty, pop u. If u is unvisited in G^T, run DFS on G^T starting from u. All vertices visited in this DFS pass form a complete SCC.

Algorithm 3: Tarjan's Single-DFS Algorithm ($O(V + E)$)

Tarjan's algorithm finds SCCs in a single DFS pass using low-link values:

1. Track discovery times `tin[u]` and maintain `low[u]`, which is the lowest discovery time reachable from u using at most one back edge.
2. Push vertices onto a helper stack as they are visited.
3. If `low[u] == tin[u]`, it means u is the root of an SCC. Pop all nodes from the stack until we reach u; these nodes form an SCC.

4.2.3 C++ Pseudocode / Implementation (Kosaraju & Tarjan)

```cpp
#include <vector>
#include <stack>
#include <algorithm>

// Kosaraju's Algorithm
```

```cpp
void kosaraju_dfs1(int u, const std::vector<std::vector<int>>& adj, std::vector<bool>& visited, std::stack<int>& s) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) kosaraju_dfs1(v, adj, visited, s);
    }
    s.push(u);
}

void kosaraju_dfs2(int u, const std::vector<std::vector<int>>& adjT, std::vector<bool>& visited, std::vector<int>& comp) {
    visited[u] = true;
    comp.push_back(u);
    for (int v : adjT[u]) {
        if (!visited[v]) kosaraju_dfs2(v, adjT, visited, comp);
    }
}

std::vector<std::vector<int>> runKosaraju(int V, const std::vector<std::vector<int>>& adj) {
    std::stack<int> s;
    std::vector<bool> visited(V, false);
    for (int i = 0; i < V; i++) {
        if (!visited[i]) kosaraju_dfs1(i, adj, visited, s);
    }

    std::vector<std::vector<int>> adjT(V);
    for (int u = 0; u < V; u++) {
        for (int v : adj[u]) adjT[v].push_back(u);
    }

    visited.assign(V, false);
    std::vector<std::vector<int>> sccs;
    while (!s.empty()) {
        int u = s.top();
        s.pop();
        if (!visited[u]) {
            std::vector<int> comp;
            kosaraju_dfs2(u, adjT, visited, comp);
            sccs.push_back(comp);
        }
    }
    return sccs;
}
```

4.3 Question 15: Graphs - Shortest Path Problem Overview: Classifications and Algorithms

4.3.1 Problem Statement

The shortest path problem consists of finding a path between two vertices in a weighted graph such that the sum of the weights of its constituent edges is minimized. The problem is categorized as follows:

- **Single-Source Shortest Path (SSSP)**: Find shortest paths from a single source vertex s to all other vertices.
- **All-Pairs Shortest Paths (APSP)**: Find shortest paths between every pair of vertices in the graph.

4.3.2 Categorization of Algorithms

The optimal choice of algorithm depends on:

1. **Graph Topology**: General vs. DAG vs. Tree.
2. **Edge Weights**: Unweighted ($w = 1$) vs. Non-negative ($w \geq 0$) vs. Negative ($w < 0$).

Algorithm	Type	Weights	Time Complexity	Space
BFS	SSSP	Unweighted ($w = 1$)	$O(V + E)$	$O(V)$
DAG Shortest Path	SSSP	Any	$O(V + E)$	$O(V)$
Dijkstra (with Min-Heap)	SSSP	Non-negative	$O((V + E) \log V)$	$O(V + E)$
Bellman-Ford	SSSP	Any	$O(V \cdot E)$	$O(V)$
Floyd-Warshall	APSP	Any	$O(V^3)$	$O(V^2)$

Table 4.1: Shortest Path Algorithms comparison

4.3.3 Choosing the Right Algorithm

- For **unweighted** graphs, BFS is optimal since it explores the graph in concentric layers of equal distance.
- On **DAGs**, topological sorting allows us to solve SSSP in $O(V + E)$ by relaxing edges in topological order. This is optimal and supports negative edge weights.
- On **general graphs with non-negative weights**, Dijkstra's algorithm is preferred because of its highly efficient priority queue relaxation.
- On **graphs containing negative weights**, Bellman-Ford must be used. It can also detect negative-weight cycles.
- For **All-Pairs Shortest Paths**, Floyd-Warshall is a simple and clean dynamic programming solution.

4.4 Question 16: Graphs - Dijkstra's Algorithm: Single-Source Shortest Paths with Trace Example

4.4.1 Concept

Dijkstra's algorithm is a greedy algorithm designed to solve the Single-Source Shortest Path (SSSP) problem on weighted graphs where all edge weights are **non-negative**.

4.4.2 Greedy Choice Property

Dijkstra's algorithm maintains a set of vertices S whose final shortest-path weights have already been determined. At each step, the algorithm greedily selects the vertex $u \notin S$ with the minimum tentative distance estimate `dist[u]`. Because edge weights are non-negative, any alternative path to u through other unvisited nodes must be longer, justifying this greedy selection.

4.4.3 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <queue>

const int INF = 1e9;
struct Edge {
    int to;
    int weight;
};

void dijkstra(int src, const std::vector<std::vector<Edge>>& adj, std::
    vector<int>& dist) {
    int V = adj.size();
    dist.assign(V, INF);
    dist[src] = 0;

    // min-heap storing pairs of (distance, vertex)
    std::priority_queue<std::pair<int, int>,
                        std::vector<std::pair<int, int>>,
                        std::greater<std::pair<int, int>>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue; // Stale state

        for (auto& edge : adj[u]) {
            int v = edge.to;
            int w = edge.weight;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

4.4.4 Trace Example

Let us find shortest paths from source 0 in a graph with vertices $\{0, 1, 2\}$ and weighted directed edges: $0 \xrightarrow{4} 1$, $0 \xrightarrow{1} 2$, $2 \xrightarrow{2} 1$.

1. Initialize: `dist = [0, INF, INF]`, Priority Queue: `[(0, 0)]`.
2. Pop `(0, 0)`. Vertex $u = 0$. Neighbors:
   - Node 1: $0 + 4 < \text{INF} \implies$ `dist[1]` $= 4$. Push `(4, 1)`.
   - Node 2: $0 + 1 < \text{INF} \implies$ `dist[2]` $= 1$. Push `(1, 2)`.

   Queue is now: `[(1, 2), (4, 1)]`.
3. Pop `(1, 2)`. Vertex $u = 2$. Neighbor 1:
   - Node 1: $1 + 2 = 3 <$ `dist[1]`$(4) \implies$ `dist[1]` $= 3$. Push `(3, 1)`.

   Queue is now: `[(3, 1), (4, 1)]`.
4. Pop `(3, 1)`. Vertex $u = 1$. No neighbors. Queue: `[(4, 1)]`.
5. Pop `(4, 1)`. Stale entry (since $4 >$ `dist[1]`(3)), skipped.
6. End of algorithm. Final shortest distances from 0: `[0, 3, 1]`.

4.5 Question 17: Graphs - Minimum Spanning Trees (MST): Prim's and Kruskal's Algorithms

4.5.1 Minimum Spanning Tree (MST)

For a connected, undirected, weighted graph $G = (V, E)$, a Minimum Spanning Tree is a subset of edges $T \subseteq E$ that connects all vertices in V without cycles, while minimizing the total sum of edge weights.

4.5.2 Greedy Spanning Tree Properties

- **Cut Property**: For any cut of the graph, the minimum weight edge crossing the cut must belong to the MST.
- **Cycle Property**: For any cycle in the graph, the maximum weight edge in the cycle cannot belong to the MST.

4.5.3 Consecutive Algorithms

We present both Prim's and Kruskal's algorithms consecutively.

Algorithm 1: Prim's Algorithm (Dense Growing Tree)

Prim's algorithm builds the tree node-by-node, starting from an arbitrary root vertex:

1. Maintain a cut with a set of visited vertices.
2. Use a priority queue to select the cheapest edge connecting a visited vertex to an unvisited vertex.
3. Add the selected vertex to the tree and repeat until all vertices are visited.

Algorithm 2: Kruskal's Algorithm (Global Edge Selection)

Kruskal's algorithm builds the spanning forest edge-by-edge:

1. Sort all edges in the graph in ascending order of their weights.
2. Initialize a Disjoint Set Union (DSU) structure to track connected components.
3. Iterate through the sorted edges (u, v). If u and v belong to different components, add (u, v) to the MST and merge their components using DSU. If they are already in the same component, skip the edge to prevent a cycle.

4.5.4 C++ Pseudocode / Implementation (Kruskal)

```cpp
#include <vector>
#include <algorithm>

struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

class DSU {
```

```
    std::vector<int> parent;
public:
    DSU(int n) {
        parent.resize(n);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int x, int y) {
        int rX = find(x), rY = find(y);
        if (rX == rY) return false;
        parent[rX] = rY;
        return true;
    }
};

std::vector<Edge> kruskalMST(int V, std::vector<Edge>& edges, int&
   total_weight) {
    std::sort(edges.begin(), edges.end());
    DSU dsu(V);
    std::vector<Edge> mst;
    total_weight = 0;

    for (const auto& edge : edges) {
        if (dsu.unite(edge.u, edge.v)) {
            mst.push_back(edge);
            total_weight += edge.w;
        }
    }
    return mst;
}
```

4.6 Question 18: Graphs - Floyd-Warshall Algorithm: All-Pairs Shortest Paths with Trace Example

4.6.1 Concept and DP Formulation

The Floyd-Warshall algorithm is a Dynamic Programming algorithm used to find the shortest paths between all pairs of vertices in a weighted graph. It can handle negative edge weights, provided there are no negative-weight cycles.

Let $D_{i,j}^{(k)}$ be the shortest path distance from vertex i to vertex j using only a subset of vertices $\{1, 2, \ldots, k\}$ as intermediate nodes.

- **Base Case**: For $k = 0$ (no intermediate nodes allowed), the distance is simply the weight of the direct edge:

$$D_{i,j}^{(0)} = w(i,j) \quad \text{if } (i,j) \in E, \quad D_{i,i}^{(0)} = 0, \quad D_{i,j}^{(0)} = \infty \quad \text{otherwise}$$

- **Recurrence Relation**:

$$D_{i,j}^{(k)} = \min\left(D_{i,j}^{(k-1)}, D_{i,k}^{(k-1)} + D_{k,j}^{(k-1)}\right)$$

4.6.2 C++ Pseudocode / Implementation

We can optimize the space complexity from $O(V^3)$ to $O(V^2)$ by updating the 2D distance matrix in-place.

```cpp
#include <vector>
#include <algorithm>

const int INF = 1e9;

void floydWarshall(int V, std::vector<std::vector<int>>& dist) {
    // dist is initially populated with edge weights, dist[i][i] = 0, and INF otherwise
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF) {
                    dist[i][j] = std::min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }
}
```

4.6.3 Trace Example

Let's trace a 3-vertex graph $\{0, 1, 2\}$ with edges: $0 \xrightarrow{4} 1$, $1 \xrightarrow{1} 2$, $0 \xrightarrow{8} 2$. Initial matrix $D^{(0)}$ ($k = 0$ starting state):

$$D^{(0)} = \begin{pmatrix} 0 & 4 & 8 \\ \infty & 0 & 1 \\ \infty & \infty & 0 \end{pmatrix}$$

1. **Step** $k = 0$ (allowing vertex 0 as intermediate): No pathways are shortened since vertex 0 has no incoming paths from other vertices. $D^{(1)} = D^{(0)}$.

2. **Step** $k = 1$ (allowing vertex 1 as intermediate): Evaluate path $0 \to 1 \to 2$:

$$\text{dist}[0][2] = \min(\text{dist}[0][2], \text{dist}[0][1] + \text{dist}[1][2]) = \min(8, 4 + 1) = 5$$

   The matrix $D^{(2)}$ becomes:

$$D^{(2)} = \begin{pmatrix} 0 & 4 & 5 \\ \infty & 0 & 1 \\ \infty & \infty & 0 \end{pmatrix}$$

3. **Step** $k = 2$ (allowing vertex 2 as intermediate): No further improvements are possible because vertex 2 has no outgoing edges.

Final all-pairs shortest paths matrix is $D^{(2)}$.

Chapter 5

Algebraic Algorithms

5.1 Question 19: Extended Euclidean Algorithm - Recursive and Iterative Formulations

5.1.1 Problem Formulation and Bézout's Identity

The standard Euclidean algorithm computes the greatest common divisor (GCD) of two integers a and b. The Extended Euclidean Algorithm also computes the integer coefficients x and y that satisfy Bézout's Identity:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

5.1.2 Consecutive Algorithms

We present both the recursive and iterative derivations consecutively.

Algorithm 1: Recursive Extended Euclidean Algorithm

Let $g = \gcd(a, b)$. The recursive step uses division with remainder: $a = q \cdot b + (a \bmod b)$, where $q = \lfloor a/b \rfloor$. By induction, the recursive call returns coefficients x_2 and y_2 such that:

$$b \cdot x_2 + (a \bmod b) \cdot y_2 = g$$

Substitute $a \bmod b = a - \lfloor a/b \rfloor \cdot b$:

$$b \cdot x_2 + (a - \lfloor a/b \rfloor \cdot b) \cdot y_2 = a \cdot y_2 + b \cdot (x_2 - \lfloor a/b \rfloor y_2) = g$$

Comparing with $a \cdot x_1 + b \cdot y_1 = g$, we get:

$$x_1 = y_2$$

$$y_1 = x_2 - \lfloor a/b \rfloor \cdot y_2$$

Algorithm 2: Iterative Extended Euclidean Algorithm

The iterative approach runs the Euclidean division forwards while tracking the coefficients using auxiliary variables, avoiding recursion stack overhead.

5.1.3 C++ Pseudocode / Implementation

```cpp
// Recursive Implementation
int extGCD_recursive(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = extGCD_recursive(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return d;
}

// Iterative Implementation
int extGCD_iterative(int a, int b, int& x, int& y) {
    int x0 = 1, y0 = 0; // coefficients for 'a'
    int x1 = 0, y1 = 1; // coefficients for 'b'
    while (b != 0) {
        int q = a / b;
        int r = a % b;
        a = b; b = r;

        int next_x = x0 - q * x1;
        int next_y = y0 - q * y1;
        x0 = x1; x1 = next_x;
        y0 = y1; y1 = next_y;
    }
    x = x0; y = y0;
    return a;
}
```

5.1.4 Trace Example

Let's find coefficients for $a = 30, b = 12$.

- Base: $\gcd(30, 12)$.
- $30/12 = 2$ remainder $6 \implies$ call $\gcd(12, 6)$.
- $12/6 = 2$ remainder $0 \implies$ call $\gcd(6, 0)$.
- Base case $b = 0 \implies \gcd = 6$, returns $x_2 = 1, y_2 = 0$.
- Backtrack to $\gcd(12, 6)$:

$$x = y_2 = 0$$

$$y = x_2 - (12/6) \cdot y_2 = 1 - 2 \cdot 0 = 1$$

- Backtrack to $\gcd(30, 12)$:

$$x = 1$$

$$y = 0 - (30/12) \cdot 1 = 0 - 2 \cdot 1 = -2$$

- Final: $\gcd(30, 12) = 6$, and $30(1) + 12(-2) = 6$.

5.2 Question 20: Algebraic Algorithms - Integer Factorization of Complexity $O(\sqrt{n})$

5.2.1 Concept

Integer factorization is the process of decomposing a composite number n into a product of smaller prime integers. It is a fundamental problem in number theory and forms the security foundation of modern cryptographic systems.

5.2.2 Trial Division Algorithm

The basic mathematical principle is that if a composite number n has a factor, it must have at least one factor $d \leq \sqrt{n}$. If all numbers up to $\sqrt{n}$ do not divide n, then n must be prime. We can optimize trial division by checking 2 first, and then only checking odd numbers up to $\sqrt{n}$ (or stepping by 6 to skip multiples of 2 and 3).

5.2.3 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <iostream>

std::vector<int> factorize(int n) {
    std::vector<int> prime_factors;
    // Step 1: Extract 2s
    while (n % 2 == 0) {
        prime_factors.push_back(2);
        n /= 2;
    }
    // Step 2: Extract odd factors up to sqrt(n)
    for (int d = 3; d * d <= n; d += 2) {
        while (n % d == 0) {
            prime_factors.push_back(d);
            n /= d;
        }
    }
    // Step 3: If n is still > 1, then remaining n must be prime
    if (n > 1) {
        prime_factors.push_back(n);
    }
    return prime_factors;
}
```

5.2.4 Complexity Analysis

The time complexity of this algorithm is $O(\sqrt{n})$ because the loop runs at most $\sqrt{n}/2$ times. The space complexity is $O(\log n)$ to store the prime factors.

5.2.5 Application of Choice: Cryptography (RSA Key Security)

In the RSA cryptosystem, the public key is a large composite number $N = p \cdot q$, where p and q are two secret prime numbers. The security of RSA relies directly on the difficulty of the integer factorization problem: if an eavesdropper can factor N to find p and q, they can calculate the private decryption key. A $O(\sqrt{n})$ trial division is efficient for small numbers but completely impractical for 2048-bit keys, making RSA cryptographically secure.

5.3 Question 21: Algebraic Algorithms - Euler's Totient Function $\phi(n)$ with $O(\sqrt{n})$ Implementation

5.3.1 Concept

Euler's Totient Function $\phi(n)$ is a number-theoretic function that counts the number of positive integers up to n that are relatively prime (coprime) to n (i.e., $\gcd(i, n) = 1$).

5.3.2 Mathematical Formula

Using the fundamental theorem of arithmetic, any integer n can be factored into distinct prime factors: $n = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$. Euler's product formula states:

$$\phi(n) = n \cdot \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right)$$

5.3.3 C++ Pseudocode / Implementation

We can compute this function in $O(\sqrt{n})$ time by performing a trial division to identify all unique prime factors of n and updating the product formula dynamically.

```cpp
int phi(int n) {
    int result = n;
    int temp = n;
    for (int p = 2; p * p <= temp; p++) {
        if (temp % p == 0) {
            // p is a prime factor
            while (temp % p == 0) {
                temp /= p;
            }
            result -= result / p; // Equivalent to result * (1 - 1/p)
        }
    }
    if (temp > 1) {
        result -= result / temp; // Final prime factor
    }
    return result;
}
```

5.3.4 Complexity Analysis

- **Time Complexity**: $O(\sqrt{n})$ as the loop executes at most $\sqrt{n}$ times.
- **Space Complexity**: $O(1)$ auxiliary space.

5.3.5 Application of Choice: Modular Multiplicative Inverse

By Euler's Theorem, if $\gcd(a, n) = 1$, then:

$$a^{\phi(n)} \equiv 1 \pmod{n} \implies a \cdot a^{\phi(n)-1} \equiv 1 \pmod{n}$$

Therefore, the modular multiplicative inverse of a modulo n can be calculated as $a^{\phi(n)-1} \bmod n$. This is heavily used in cryptographic applications when the modulus is not a prime.

5.4 Question 22: Algebraic Algorithms - Modular Arithmetic and Binary Modular Exponentiation

5.4.1 Modular Arithmetic and Congruence

Two integers a and b are said to be congruent modulo m if their difference $a - b$ is divisible by m. We write:

$$a \equiv b \pmod{m}$$

Modular arithmetic preserves standard arithmetic operations:

- $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

Division is not defined directly. Instead, we multiply by the modular inverse: $a/b \equiv a \cdot b^{-1} \pmod{m}$.

5.4.2 Modular Exponentiation (Binary Exponentiation)

To compute $a^b \bmod m$ for large exponents b, a naive multiplication takes $O(b)$ multiplications, which is highly inefficient. Instead, we use **Binary Exponentiation** (Repeated Squaring) based on the binary representation of b:

$$a^b = \begin{cases} 1 & \text{if } b = 0 \\ (a^{b/2})^2 & \text{if } b \text{ is even} \\ a \cdot (a^{(b-1)/2})^2 & \text{if } b \text{ is odd} \end{cases}$$

5.4.3 C++ Pseudocode / Implementation

```cpp
// Iterative Binary Modular Exponentiation
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) { // If exponent is odd
            res = (res * base) % mod;
        }
        base = (base * base) % mod; // Square the base
        exp >>= 1;                  // Divide exponent by 2
    }
    return res;
}
```

5.4.4 Complexity for Large Digits

When working with extremely large numbers (such as 1024-bit integers in RSA), the operations are performed on B-digit BigInt numbers:

- **Addition / Subtraction**: $O(B)$ time.
- **Multiplication**: $O(B^2)$ using schoolbook multiplication, $O(B^{\log_2 3}) \approx O(B^{1.58})$ using Karatsuba, or $O(B \log B \log \log B)$ using Schönhage-Strassen (FFT-based multiplication).
- **Exponentiation**: $O(\log e)$ multiplications of B-digit numbers.

Chapter 6

String Algorithms

6.1 Question 23: Niske - Polynomial Rolling Hash Function, Collisions, and Applications

6.1.1 Concept of String Hashing

String hashing is a technique that maps a string of arbitrary length to a fixed-size integer. This allows us to compare strings in $O(1)$ time by comparing their hash values, rather than performing a character-by-character comparison which takes $O(L)$ time.

6.1.2 Polynomial Rolling Hash Function

To hash a string $S = s[0..L-1]$, we treat its characters as coefficients of a polynomial evaluated at a prime base p modulo a large prime m:

$$H(S) = \left(\sum_{i=0}^{L-1} s[i] \cdot p^i \right) \bmod m$$

- **Base** p: Must be a prime greater than the alphabet size. Typically $p = 31$ for lowercase English letters, or $p = 53$ for mixed-case.
- **Modulus** m: Must be a large prime to minimize collisions, typically $m = 10^9 + 7$ or $m = 10^9 + 9$.

6.1.3 Collision Probability Reduction

A collision occurs when two different strings produce the same hash value: $H(A) = H(B)$. According to the Birthday Paradox, the probability of collisions increases with more strings. To significantly reduce this probability, we use **Double Hashing**: we calculate two independent hashes using two different prime moduli m_1, m_2 and bases p_1, p_2. The combined hash is the pair (H_1, H_2). This reduces the probability of a collision to approximately $O(1/(m_1 \cdot m_2)) \approx 10^{-18}$, which is cryptographically secure.

6.1.4 Applications

- **Rabin-Karp Pattern Matching**: Fast sliding-window substring search.
- **Comparing Substrings in** $O(1)$: By precomputing prefix hashes and powers of p:

$$H(s[i..j]) = \left((Pref[j+1] - Pref[i]) \cdot p^{-i} \right) \bmod m$$

- **Determining Number of Unique Substrings**.

6.2 Question 24: Niske - Rabin-Karp Substring Search with Trace Example

6.2.1 Rabin-Karp Concept

The Rabin-Karp algorithm searches for a pattern P of length M in a text T of length N by sliding a window of size M over T. Instead of comparing characters, it computes the polynomial rolling hash of the pattern and compares it to the rolling hash of the current window.

6.2.2 Rolling Hash Optimization

A naive recomputation of the window hash takes $O(M)$ time. Rabin-Karp uses a rolling hash to update the hash in $O(1)$ time. When shifting the window from $T[i..i+M-1]$ to $T[i+1..i+M]$, we subtract the leading character and append the trailing character:

$$H_{\text{next}} = \left((H_{\text{prev}} - T[i] \cdot p^{M-1}) \cdot p + T[i+M]\right) \bmod m$$

6.2.3 C++ Pseudocode / Implementation

```cpp
#include <string>
#include <vector>
#include <iostream>

void rabin_karp(const std::string& S, const std::string& T) {
    int M = S.length();
    int N = T.length();
    long long p = 31;
    long long m = 1e9 + 9;

    long long p_pow = 1;
    for (int i = 0; i < M - 1; i++) {
        p_pow = (p_pow * p) % m;
    }

    long long h_S = 0, h_T = 0;
    for (int i = 0; i < M; i++) {
        h_S = (h_S * p + S[i]) % m;
        h_T = (h_T * p + T[i]) % m;
    }

    for (int i = 0; i <= N - M; i++) {
        if (h_S == h_T) {
            // Spurious hit verification
            if (T.substr(i, M) == S) {
                std::cout << "Pattern found at index " << i << std::
                    endl;
            }
        }
        if (i < N - M) {
            h_T = (h_T - T[i] * p_pow) % m;
            h_T = (h_T * p + T[i + M]) % m;
            h_T = (h_T + m) % m; // Ensure positive
        }
    }
}
```

6.2.4 Trace Example

Let's search for pattern $P =$ `"ab"` in text $T =$ `"cab"`. Let $p = 3, m = 11$.

- Character values: `'a'` $= 1$, `'b'` $= 2$, `'c'` $= 3$.
- Pattern hash: $H(P) = (1 \cdot 3 + 2) \bmod 11 = 5$.
- Initial window $T[0..1] =$ `"ca"`:

$$H(T[0..1]) = (3 \cdot 3 + 1) \bmod 11 = 10 \quad (\neq 5)$$

- Slide to window $T[1..2] =$ `"ab"`:

$$H(T[1..2]) = \big((10 - T[0] \cdot p^1) \cdot p + T[2]\big) \bmod 11$$

$$= ((10 - 3 \cdot 3) \cdot 3 + 2) \bmod 11 = (1 \cdot 3 + 2) \bmod 11 = 5$$

- Hash matches ($5 == 5$). Compare strings: $T[1..2] == P \implies$ `"ab" == "ab"`. Match found at index 1!

6.3 Question 25: Niske - Z-Algorithm and KMP Pattern Matching Algorithms

6.3.1 Concept

Pattern matching involves finding all occurrences of a pattern P of length M in a text T of length N. As requested, we present both optimal $O(N + M)$ algorithms consecutively.

6.3.2 Algorithm 1: The Z-Algorithm

The Z-Algorithm computes a **Z-array** of size L for a concatenated string $S = P + \texttt{"\$"} + T$. An entry $Z[i]$ is the length of the longest common prefix between S and the suffix of S starting at index i. Whenever $Z[i] == M$, we have found a match of pattern P at index $i - M - 1$ in the text. The algorithm maintains a sliding window $[L, R]$ representing the rightmost substring matching a prefix of S, allowing $O(1)$ updates using dynamic programming.

6.3.3 Algorithm 2: Knuth-Morris-Pratt (KMP) Algorithm

The KMP algorithm avoids backtracking in the text by precomputing a prefix-suffix table π (or LSP array) for the pattern P. An entry $\pi[i]$ is the length of the longest proper prefix of $S[0..i]$ that is also a suffix of $S[0..i]$. If a character mismatch occurs at index j of the pattern, we slide the pattern to align with the prefix of length $\pi[j - 1]$, skipping redundant comparisons.

6.3.4 C++ Pseudocode / Implementation

```cpp
#include <string>
#include <vector>
#include <iostream>

// 1. Z-Algorithm
std::vector<int> compute_z(const std::string& s) {
    int n = s.length();
    std::vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = std::min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

// 2. KMP Algorithm
std::vector<int> compute_prefix_function(const std::string& p) {
    int m = p.length();
    std::vector<int> pi(m, 0);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j]) j = pi[j - 1];
        if (p[i] == p[j]) j++;
        pi[i] = j;
    }
    return pi;
```

```
}
```

6.3.5 Trace Examples

Let pattern $P =$ `"aba"`.

1. **Z-Algorithm Trace** on concatenated $S =$ `"aba$abacaba"`:
   - $Z = [0, 0, 1, 0, 3, 0, 1, 0, 3, 0, 1]$.
   - Entries $Z[4] = 3$ and $Z[8] = 3$ match $M = 3$, revealing pattern occurrences at indices 0 and 4 in text $T =$ `"abacaba"`.
2. **KMP π-array Trace** on $P =$ `"aba"`:
   - $\pi[0] = 0$.
   - $\pi[1] = 0$ (proper prefixes of `"ab"` are `"a"`, suffixes are `"b"`; no match).
   - $\pi[2] = 1$ (proper prefixes of `"aba"` are `"a", "ab"`, suffixes are `"a", "ba"`; `"a"` matches).

Chapter 7

Geometric Algorithms

7.1 Question 26: Geometrijski algoritmi - Skalarni i vektorski proizvod i primene

7.1.1 Mathematical Definitions

Let $\vec{u} = (x_1, y_1)$ and $\vec{v} = (x_2, y_2)$ be two vectors in 2D space.

1. **Skalarni proizvod (Dot Product)**:

$$\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2 = \|\vec{u}\| \|\vec{v}\| \cos\theta$$

2. **Vektorski proizvod (Cross Product in 2D)**: The 2D cross product of $\vec{u}$ and $\vec{v}$ is defined as the determinant:

$$\vec{u} \times \vec{v} = x_1 y_2 - y_1 x_2 = \|\vec{u}\| \|\vec{v}\| \sin\theta$$

7.1.2 Geometric Interpretations

- The **Dot Product** is positive if the angle θ between vectors is acute ($< 90°$), zero if orthogonal ($= 90°$), and negative if obtuse ($> 90°$). It measures projections.
- The magnitude of the **Cross Product** is equal to the signed area of the parallelogram spanned by the two vectors.

7.1.3 Core Geometric Applications

- **Determining Orientation**: The sign of the cross product indicates whether $\vec{v}$ lies to the left (CCW, positive) or to the right (CW, negative) of $\vec{u}$.
- **Orthogonality Test**: Checking if $\vec{u} \cdot \vec{v} = 0$.
- **Collinearity Test**: Checking if $\vec{u} \times \vec{v} = 0$.

7.2 Question 27: Geometrijski algoritmi - Računanje površine trougla i mnogougla i primene

7.2.1 Triangle Area

1. **Heron's Formula (Using Side Lengths)**:

$$A = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

2. **Cross Product (Using Coordinates)**: Given vertices A, B, C, the area is half the magnitude of the cross product of vectors $\vec{AB}$ and $\vec{AC}$:

$$\text{Area} = \frac{1}{2}|x_A(y_B - y_C) + x_B(y_C - y_A) + x_C(y_A - y_B)|$$

7.2.2 Arbitrary Simple Polygon Area (Gauss's Shoelace Formula)

For any simple polygon with N vertices ordered consecutively $P_0, P_1, \ldots, P_{N-1}$ where $P_i = (x_i, y_i)$, the area is computed using the Shoelace formula:

$$\text{Area} = \frac{1}{2}\left|\sum_{i=0}^{N-1}(x_i y_{i+1} - x_{i+1} y_i)\right|$$

where $P_N = P_0$.

7.2.3 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <cmath>

struct Point {
    double x, y;
};

double polygonArea(const std::vector<Point>& poly) {
    int n = poly.size();
    double area = 0.0;
    for (int i = 0; i < n; i++) {
        int next = (i + 1) % n;
        area += poly[i].x * poly[next].y - poly[next].x * poly[i].y;
    }
    return std::abs(area) / 2.0;
}
```

7.3 Question 28: Geometrijski algoritmi - Utvrivanje orijentacije trojke tačaka u ravni i njene primene

7.3.1 Orientation Test

Given three points in a 2D plane $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$, and $P_3 = (x_3, y_3)$, we wish to determine the orientation of the ordered triplet (P_1, P_2, P_3). By calculating the signed cross product of vectors $\vec{P_1P_2}$ and $\vec{P_2P_3}$, we define the orientation value Δ:

$$\Delta = (x_2 - x_1)(y_3 - y_2) - (y_2 - y_1)(x_3 - x_2)$$

7.3.2 Mathematical Characterization

- $\Delta > 0$: The orientation is **Counter-Clockwise** (CCW, left turn).
- $\Delta < 0$: The orientation is **Clockwise** (CW, right turn).
- $\Delta = 0$: The three points are **Collinear**.

7.3.3 C++ Pseudocode / Implementation

```
struct Point {
    int x, y;
};

int orientation(Point p1, Point p2, Point p3) {
    int val = (p2.x - p1.x) * (p3.y - p2.y) - (p2.y - p1.y) * (p3.x -
        p2.x);
    if (val == 0) return 0;  // Collinear
    return (val > 0) ? 1 : 2; // 1: CCW, 2: CW
}
```

7.3.4 Applications

- **Line Segment Intersection**: Two segments AB and CD intersect if and only if the orientation pairs (A, B, C) and (A, B, D) have different signs, and (C, D, A) and (C, D, B) have different signs.
- **Convex Hull Construction**: Determining whether a new point maintains a left-turn boundary.
- **Point-in-Polygon Tests**.

7.4 Question 29: Geometrijski algoritmi - Utvrivanje da li tačka pripada proizvoljnom prostom mnogouglu

7.4.1 Ray Casting Method

To determine if a point $P = (x_p, y_p)$ lies inside a simple polygon, we cast an infinite horizontal ray to the right from P:

$$\text{Ray}(t) = (x_p + t, y_p), \quad t \geq 0$$

We count the number of times this ray intersects the boundary edges of the polygon.

- If the intersection count is **odd**, the point is **inside** the polygon.
- If the intersection count is **even**, the point is **outside** the polygon.

7.4.2 Winding Number Method

An alternative approach computes the sum of the signed angles subtended by each polygon edge at point P. If the sum is non-zero (specifically 2π), the point is inside.

7.4.3 C++ Pseudocode / Implementation

Below is the robust Ray Casting algorithm.

```cpp
#include <vector>

struct Point {
    double x, y;
};

bool isInside(const std::vector<Point>& polygon, Point p) {
    int n = polygon.size();
    bool inside = false;
    for (int i = 0, j = n - 1; i < n; j = i++) {
        // Check if horizontal ray intersects the edge
        if (((polygon[i].y > p.y) != (polygon[j].y > p.y)) &&
            (p.x < (polygon[j].x - polygon[i].x) * (p.y - polygon[i].y)
                / (polygon[j].y - polygon[i].y) + polygon[i].x)) {
            inside = !inside;
        }
    }
    return inside;
}
```

7.4.4 Complexity Analysis

The time complexity is $O(N)$, where N is the number of vertices in the polygon, because we check every edge exactly once. The space complexity is $O(1)$.

7.5 Question 30: Geometrijski algoritmi - Konstrukcija prostog mnogougla i analiza složenosti

7.5.1 Problem Statement

Given a set of N points in a 2D plane, we wish to construct a simple (non-self-intersecting) polygon that has these points as its vertices.

7.5.2 Sorting Algorithm

A simple and elegant algorithm works as follows:

1. Find the point P_0 with the lowest y-coordinate (and leftmost x in case of ties). This point is guaranteed to be a vertex on the boundary of the simple polygon.
2. For all other points P_i, compute their polar angle with respect to P_0.
3. Sort the points in ascending order of their polar angles. If two points have the same angle, sort them by distance to P_0.
4. Connecting the points in this sorted order, and connecting the last point back to P_0, yields a simple, non-self-intersecting polygon.

7.5.3 C++ Pseudocode / Implementation

```cpp
#include <vector>
#include <algorithm>
#include <cmath>

struct Point {
    int x, y;
};

Point pivot;

int distSq(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) + (p1.y - p2.y) * (p1.y - p2.y);
}

// 0: Collinear, 1: CCW, 2: CW
int orientation(Point p1, Point p2, Point p3) {
    int val = (p2.x - p1.x) * (p3.y - p2.y) - (p2.y - p1.y) * (p3.x - p2.x);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}

bool compare(Point p1, Point p2) {
    int o = orientation(pivot, p1, p2);
    if (o == 0) return distSq(pivot, p1) < distSq(pivot, p2);
    return o == 1; // CCW sorted
}

void constructSimplePolygon(std::vector<Point>& points) {
    // Step 1: Find pivot
    int min_idx = 0;
```

```
    for (int i = 1; i < points.size(); i++) {
        if (points[i].y < points[min_idx].y ||
           (points[i].y == points[min_idx].y && points[i].x < points[
              min_idx].x)) {
            min_idx = i;
        }
    }
    std::swap(points[0], points[min_idx]);
    pivot = points[0];

    // Step 2: Sort based on polar angle
    std::sort(points.begin() + 1, points.end(), compare);
}
```

7.5.4 Complexity Analysis

- **Time Complexity**: $O(N \log N)$ dominated entirely by the sorting of the points. Finding the pivot takes $O(N)$ time.
- **Space Complexity**: $O(1)$ auxiliary space if sorted in-place.

7.6 Question 31: Geometrijski algoritmi - Konstrukcija konveksnog omotača: Graham Scan i Monotone Chain

7.6.1 Convex Hull Definition

The Convex Hull of a set of 2D points is the smallest convex polygon that contains all points within its boundary. It can be visualized as the shape formed by a tight rubber band surrounding all points.

7.6.2 Consecutive Algorithms

We present both Graham Scan and Andrew's Monotone Chain algorithms consecutively.

Algorithm 1: Graham Scan

Graham Scan constructs the convex hull using a polar-angle sort:

1. Find the bottom-left point P_0.
2. Sort the remaining points by polar angle with respect to P_0 (similar to Question 30).
3. Initialize a stack containing P_0, P_1, and P_2.
4. For each subsequent point P_i, check the orientation of the top two elements of the stack and P_i. While the turn is not Counter-Clockwise (CW or Collinear), pop from the stack. Push P_i to the stack.

Algorithm 2: Andrew's Monotone Chain Algorithm

Andrew's Monotone Chain algorithm constructs the hull by sorting lexicographically:

1. Sort the points lexicographically by x-coordinate, and then by y-coordinate.
2. Build the **Lower Hull** by iterating left-to-right, popping points from the stack if they do not form a Counter-Clockwise turn.
3. Build the **Upper Hull** by iterating right-to-left, performing the same check.
4. Combine both hulls (removing duplicate endpoints) to obtain the full convex hull.

7.6.3 C++ Pseudocode / Implementation (Monotone Chain)

```cpp
#include <vector>
#include <algorithm>

struct Point {
    long long x, y;
    bool operator<(const Point& other) const {
        if (x != other.x) return x < other.x;
        return y < other.y;
    }
};

long long cross_product(Point o, Point a, Point b) {
    return (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
}

```

```
std::vector<Point> convex_hull(std::vector<Point>& pts) {
    int n = pts.size(), k = 0;
    if (n <= 3) return pts;
    std::vector<Point> hull(2 * n);

    std::sort(pts.begin(), pts.end());

    // Lower Hull
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && cross_product(hull[k-2], hull[k-1], pts[i]) <=
            0) k--;
        hull[k++] = pts[i];
    }

    // Upper Hull
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross_product(hull[k-2], hull[k-1], pts[i]) <=
            0) k--;
        hull[k++] = pts[i];
    }

    hull.resize(k - 1);
    return hull;
}
```

7.6.4 Complexity Analysis

Both algorithms have a time complexity of $O(N \log N)$ due to sorting. The linear scan takes $O(N)$ because each point is pushed and popped at most once.